Exercise 2: Write Better Prompts

Session: 2 — Bug Fixing & Effective Prompting
Duration: 35 minutes, plus a 5-minute closing round
Project: The same BookStore API. Some of its tests fail on purpose.

Goal

You write three prompts with the techniques from the slides. A coach checks each prompt before it runs. Then you compare the result with what the coach predicted.

The goal is a better prompt, not a fixed bug. If a bug stays unfixed but you can name what was missing from your prompt, you have learned more than someone who got lucky.

Start here

- Open a terminal in the `bookstore` folder and start `claude`. Keep this one session for tasks 1 and 2. Task 3 tells you when to restart.
- Keep an empty text file open for your notes, outside `bookstore`.
- Training mode is on: before Claude explains or fixes a bug, it asks you one question. Never edit `CLAUDE.md`. Sessions 3 to 8 need it.
- Stuck, or out of time? Type *just tell me*.

Tasks

1. A prompt that writes `docs/orientation.md`

You produce the file `docs/orientation.md`, next to `main.py`. The `docs/` folder does not exist yet.

Write one prompt that produces a short guide for a new developer, in two parts:

1. **Module tree**: one line per module, saying what that module is responsible for.
2. **Request flow**: the path of one request, from `main.py` to the database and back. One line per step, each with a `file:line` reference, written like `bookstore/handler/book.py:42`.

Tasks 1 and 2 follow the coach loop from the slides: **Draft → Coach → Revise → Ship → Debrief.**

Steps:

1. **Draft your prompt**. Use these techniques: CONTEXT-TASK-OUTCOME, role framing, Scope it, Direct it, Define done, `@file`, and examples. One of the seven adds nothing here; the closing round asks which one. **Your prompt must say: write the result to `docs/orientation.md`, and create the folder.** Without that line, the sub-agent only reports in the chat.
2. **Coach and revise**. Run `/prompt-coach 1` (1 is the task number), then paste your draft as your next message. The coach checks it and asks one question. Revise until it says the prompt is ready: at most two rounds. Disagree with the coach? Type *run it anyway*, without quotes.
3. **Save your prompt, then ship**. Copy your final prompt into your notes. Then type *ship it*. The coach sends your prompt, word for word, to a sub-agent: a second Claude session that knows only your prompt. Read task 2 while the sub-agent works.
4. **Debrief**. The sub-agent's report comes back into your chat. Read it with the coach: did its predictions come true? Then open `docs/orientation.md` and check two `file:line` references. Do they point at the line they name?

The prompt on the slide *Let's do a Prompt Analysis* has the same four parts: the technology, where to start, what to deliver, and proof for every statement.

Done when: `docs/orientation.md` exists, and every step in the request flow has a `file:line` reference.

2. A prompt that asks for failing tests first

You produce two new test cases in `tests/util/test_pagination.py` and a fix in `bookstore/util/pagination.py`.

`paginate()` breaks when `page` is `0` or negative. No test covers those cases yet. Decide first: what *should* happen when `page` is `0`? Page 1 again, or an error: both are fine. Write your choice into your prompt. You decide this, not Claude. Session 6 builds on your choice.

Steps:

1. **Draft your prompt**. Techniques you need here: Scope it, Direct it (say "tests first" in those words), Examples, Constrain it, Define done. For Examples, point at the existing test function `test_table` in `tests/util/test_pagination.py`. Your new cases belong in that same style. Ask for both test runs in the report: before the fix and after.
2. **Coach and revise**. Run `/prompt-coach 2` and paste your draft.

3. **Ship and test**. Type *ship it*. Then run `python3 -m unittest tests.util.test_pagination` yourself. The sub-agent reports only what your prompt asked for.

The better prompt on the slide *Which Prompt is More Effective?* named the function, said no mocks, and asked for a failing test before the fix.

When the trainer calls task 3, move to task 3, even if this task is not finished.

Done when: the report shows the new tests failing *before* the fix and passing *after*, and `python3 -m unittest tests.util.test_pagination` passes.

3. A prompt for plan mode

You fix `test_create_review_nonexistent_book` in `tests/handler/test_review.py`. It expects `404` but gets `201`. The review flow runs from `bookstore/handler/review.py` to `bookstore/store/review.py`, and the fix can go in more than one place. That makes it a job for plan mode: Claude proposes a plan, you approve it, then it writes code (*Plan Mode: Explore Before Editing*).

This time you do not type *ship it*. You run the prompt yourself, in a fresh session that has not seen the coach's advice.

Steps:

1. **Copy the error**. Run `python3 -m unittest tests.handler.test_review`. Copy the output of `test_create_review_nonexistent_book` into your notes, exactly as printed.
2. **Draft your planning prompt**. Techniques you need here: error context (the paste from step 1), plan mode, directed thinking, Scope it, Define done. Directed thinking means: write in the prompt what the plan must think about (*Prompting & Extended Thinking*). Ask for one recommendation with reasons, not a list of options. Do not start this prompt with the experiment prefix.
3. **Coach and revise**. Run `/prompt-coach 3` and paste your draft. When the coach says it is ready, copy the prompt into your notes.
4. **Run it in plan mode**. Close Claude and start it again with `claude`. The coach is gone now; you do not need it again. Press Shift+Tab until the screen says plan mode. Paste your prompt and read the plan. Training mode is on, so Claude may ask one question before it plans. Answer it in one line.
5. **Question one step, then approve**. Pick one step. Ask Claude why that step, and what breaks if it is wrong. Write the step and the answer in your notes. When Claude asks whether to go ahead, choose the *Yes* answer that accepts the edits

automatically. Claude leaves plan mode and writes the code. Run `python3 -m unittest tests.handler.test_review`.

Done when: your notes hold the plan step you questioned and Claude's answer, and `test_create_review_nonexistent_book` passes.

4. Closing: make the tests pass, then commit

Sessions 3 to 8 build on this code, so every test should pass at the end.

1. **Fix the two book tests**. `test_create_book_returns_201` and `test_delete_book_returns_204` still fail. The prompt below is written for you. Run `python3 -m unittest tests.handler.test_book`, paste the output of those two tests at the marked place, and send the whole text to Claude. The first line is an experiment prefix, like in session 1.

```
[Exercise 2 experiment — execute directly, no leading questions.]
Two tests in tests/handler/test_book.py fail. The bug is in bookstore/handler/book.py. Fix
the handlers. Do not change any test file. This is the exact test output:

<paste the output of the two failing tests here>

Done when `python3 -m unittest tests.handler.test_book` passes. Show me that test run.
```

   Read the prompt once more. The paste is error context (*Providing Error Context*). The file name is Scope it. "Do not change any test file" is Constrain it. The last line is Define done.

2. **Check the validation test**. If `test_create_review_validation` fails too, send this:

```
[Exercise 2 experiment — execute directly, no leading questions.]
Add input validation to create_review in bookstore/handler/review.py: rating must be 1-5,
review_text must be between 10 and 500 characters
```

3. **Catch up and commit**. Whatever state you are in: press Shift+Tab until the screen says auto mode, then run `/catch-up 2`. It fixes the tests that still fail, in the project's own style, and commits. Leave it running while the trainer starts session 3. If its report says it took a decision for you, write that decision down.

Done when: `/catch-up 2` is running, and your notes name every failing test and the technique your prompt was missing.

Bonus (only if time remains)

The same review, with and without a role. Send these two prompts, with `/clear` before each one. Does the role change what Claude finds (*Role Framing*)?

```
[Exercise 2 experiment — execute directly, no leading questions.]
Review bookstore/store/book.py.
```

```
[Exercise 2 experiment — execute directly, no leading questions.]
Review bookstore/store/book.py as a database performance specialist. Focus on the number of
queries.
```

Closing round

The trainer asks the group. Have these answers ready:

- Task 1 listed seven techniques. Which one added nothing there, and why?
- Which single sentence in one of your prompts helped the most?
- Which missing technique cost you something? Did the coach predict it?
- Where did you disagree with the coach, and were you right?

After the closing round

Run `/verify-exercise 2` in any Claude session. It asks for your task 1 prompt from your notes and checks `docs/orientation.md` against it, one line at a time.